

P2615R0: Meaningful exports

Audience: CWG

S. Davis Herring <herring@lanl.gov>

July 1, 2022

Introduction

This paper addresses [CWG2443](#) by forbidding applying export directly to certain meaningless declarations. Conversely, it also allows all kinds to appear with export `{}`. For consistency extern "C" is given the same restrictions when used directly.

Approach

The expansion of permissions here was originally proposed in [P1766](#), which SG2 considered in Cologne but which has never been seen by EWG (perhaps because the rest of it was considered “accepted”). EWG did [see the corresponding NB comment](#), but the vote was too close to progress it. The only apparent new information on the subject is that the wording does not specify a coherent set of restrictions (as opposed to finding the issues that some imagined might surface with the general direction). As such, this wording does make the change (largely by separating the relevant candidates in the *declaration* grammar); EWG approved the design change (while it had the form of core issue drafting).

Wording

Relative to N4910.

#[basic]

#[basic.pre]

Change paragraph 5:

- Every name is introduced by a *declaration*, which is a

1. `name-declaration`,
2. `block-name-declaration`, or

[...]

#[basic.def]

Change bullet (2.20):

- it is an explicit specialization ([temp.expl.spec]) whose `name-declaration` is not a definition.

#[basic.lookup.unqual]

Change paragraph 1:

- [...]

~~[Note: A using-directive is exported if and only if it appears in a header unit. — end note]~~

#[stmt.dcl]

Change the grammar in paragraph 1:

- *declaration-statement*:

block-name-declaration
block-special-declaration

#[dcl.dcl]

#[dcl.pre]

Change the grammar in paragraph 1:

- [...]

declaration:

name-declaration
special-declaration

name-declaration:

block-name-declaration
nodeclspec-function-declaration
function-definition
template-declaration
linkage-specification
namespace-definition

special-declaration:

block-special-declaration
deduction-guide
explicit-instantiation
explicit-specialization
export-declaration
linkage-specification
namespace-definition
empty-declaration
attribute-declaration
module-import-declaration

block-name-declaration:

simple-declaration
asm-declaration
namespace-alias-definition
using-declaration
using-enum-declaration
using-directive
static-assert-declaration
alias-declaration
opaque-enum-declaration

block-special-declaration:

asm-declaration
static-assert-declaration

[...]

#[dcl.meaning.general]

Change paragraph 1:

- A declarator contains exactly one *declarator-id*; it names the entity that is declared. If the *unqualified-id* occurring in a *declarator-id* is a *template-id*, the declarator shall appear in the *name-declaration* of a *template-declaration* ([temp.decls]), *explicit-specialization* ([temp.expl.spec]), or *explicit-instantiation* ([temp.explicit]).

[...]

#[dcl.link]

Change the grammar in paragraph 2:

- *linkage-specification*:

```
extern string-literal { declaration-seqopt }  
extern string-literal name-declaration
```

Change paragraph 4:

- ~~A module-import-declaration shall not be directly contained in a linkage-specification.~~ A *module-import-declaration* appearing in a linkage specification with other than C++ language linkage is conditionally-supported with implementation-defined semantics.

#[module.interface]

Change the grammar before paragraph 1:

- *export-declaration*:

```
export name-declaration  
export { declaration-seqopt }  
export-keyword module-import-declaration
```

Change paragraph 1:

- An *export-declaration* shall inhabit a namespace scope and appear in the purview of a module interface unit. An *export-declaration* shall not appear directly or indirectly within an unnamed namespace or a *private-module-fragment*. An *export-declaration* has the declarative effects of its *name-declaration*, *declaration-seq* (if any), or *module-import-declaration*. *The name-declaration of an export-declaration shall not declare a partial specialization ([temp.decls.general]).* ~~The declaration or declaration-seq of an export-declaration shall not contain an export-declaration or module-import-declaration.~~

[Note: An *export-declaration* does not establish a scope. — end note]

Change paragraph 3:

- ~~An exported declaration that is not a module-import-declaration shall declare at least one name.~~ If ~~the~~ *an exported* declaration is not within a header unit, it shall not declare a name with internal linkage.

Change paragraph 4:

- [...]

```
export module M;  
export namespace {}           // error: does not introduce any names namespace has internal linkage  
export namespace {  
    int a1;                    // error: export of name with internal linkage  
}  
namespace {  
    export int a2;             // error: export of name with internal linkage  
}  
export static int b;          // error: b explicitly declared static  
export int f();               // OK
```

```
export namespace N { }           // OK
export using namespace N;       // error: does not declare a nameOK
```

[...]

#[temp]

#[temp.explicit]

Change the grammar in paragraph 2:

- *explicit-instantiation*:

```
externopt template name-declaration
```

Change paragraph 4:

- If the explicit instantiation is for a class or member class, the *elaborated-type-specifier* in the *name*-*declaration* shall include a *simple-template-id*; otherwise, the *name*-*declaration* shall be a *simple-declaration* whose *init-declarator-list* comprises a single *init-declarator* that does not have an *initializer*. If the explicit instantiation is for a variable template specialization, the *unqualified-id* in the *declarator* shall be a *simple-template-id*. [Example:

[...]

— *end example*]

Change paragraph 5:

- [...] If the *name*-*declaration* of the explicit instantiation names an implicitly-declared special member function ([special]), the program is ill-formed.

Change paragraph 6:

- The *name*-*declaration* in an *explicit-instantiation* and the *declaration* produced by the corresponding substitution into the templated function, variable, or class are two declarations of the same entity.

[Note: [...] — *end note*]

Despite its syntactic form, the *name*-*declaration* in an *explicit-instantiation* for a variable is not itself a definition and does not conflict with the definition instantiated by an explicit instantiation definition for that variable.

#[temp.expl.spec]

Change paragraph 2:

- The *declaration* in an *explicit-specialization* shall not be an *export-declaration*. An explicit specialization shall not use a *storage-class-specifier* ([dcl.stc]) other than `thread_local`.